

A

Research Project Report titled
**“HYBRID MACHINE
LEARNING BASED
FRAUD DETECTION
SYSTEM”**

Submitted to
Department of Data Science

BUNTS SANGHA’S
**S.M. SHETTY COLLEGE OF SCIENCE, COMMERCE &
MANAGEMENT STUDIES (AUTONOMOUS), POWAI,
MUMBAI-76**



For Partial Fulfillment for Degree of B.Sc. (Data Science)

2026 – 2027

In the Subject Head: Research Project

Semester V

Submitted by

Vivek Swapnaketu Puspalak

SEAT NO: _____

Under the guidance of
Asst.Prof Aishwarya Mishra

**BUNTS SANGHA'S
S.M. SHETTY COLLEGE OF SCIENCE, COMMERCE &
MANAGEMENT STUDIES (AUTONOMOUS)**



Research Project Title

“Hybrid ML based Fraud Detection System”

CLASS: T.Y.B.Sc.D.S (2026-2027)

SEMESTER : V

SUBMITTED BY (Name):

SEAT NUMBER:

**SUBMITTED TO
DEPARTMENT OF DATA SCIENCE**

**Asst.Prof Aishwarya Mishra
Project Guide**

**Dr.Tushar Sambare
Coordinator**

External Examiner

**For Partial Fulfilment for Degree of
Bachelor of Science in DS (*Data Science*)
in 2026-2027**

Declaration

I hereby declare that the research project report entitled "**Hybrid ML Based Fraud Detection System**" submitted by me for the partial fulfillment of the degree of B.Sc. Data Science is my original work and has not been submitted earlier to any other university, institution, or organization for the award of any degree, diploma, or fellowship.

The empirical findings, synthetic dataset creation, machine learning models, and system architecture documented in this report are the result of my own research and effort under the guidance of my project supervisor. All external sources of information, research papers, and technical literature have been duly acknowledged and cited in the References section.

Date: 05-10-2026

Signature:_____

Name:_____

Roll Number: ____

Acknowledgement

I would like to express my profound gratitude to everyone who supported and guided me throughout the course of this research project. The successful completion of this project, "**Hybrid ML based Fraud Detection System**," would not have been possible without their invaluable assistance.

First and foremost, I would like to extend my deepest appreciation to my project guide, **Asst.Prof Aishwarya Mishra**, for their continuous encouragement, expert guidance, and constructive feedback. Their deep knowledge of Data Science and machine learning played a pivotal role in helping me navigate the technical complexities of this project.

My sincere thanks go to the faculty members of the Data Science department for equipping me with the foundational knowledge required to undertake a project of this scale.

Lastly, I would like to thank my parents, family, and friends for their unwavering moral support, patience, and motivation during the challenging phases of system development and model training.

Date: 05-10-2026

Signature:_____

Name:_____

Roll Number: _____

Table of Contents

Sr.No	Section/Chapter
1.	Abstract
2.	Introduction
3.	Problem Statement
4.	Objective of the Study
5.	Scope and Limitations
6.	Technology Used
6.1	Programming Language
6.2	Libraries and Framework
6.3	Database and Data storage
7.	Review of Literature
7.1	Traditional Rule-Based Method
7.2	Machine Learning in Fraud Detection
7.3	The Need for Hybrid Approach
8.	Research Gap
9.	Proposed Solution
10.	Methodology and System Architecture
10.1	Data Synthesis and Simulation
10.2	Exploratory Data Analysis(EDA)
10.3	Feature Engineering
10.4	Model Training and Evaluation
10.5	System Architecture and Integration
10.6	UI- Driven Continuous Training Pipeline(MLOps)
11.	Dataset Description and Data Collection
11.1	Data Collection and Engineering Methodology
11.2	Dataset Architecture and Sample Data

12.	Data Preprocessing and Exploratory Data Analysis(EDA)
12.1	Data Preprocessing and Ingestion
12.2	Univariate Analysis
12.3	Bivariate Analysis
12.4	Multivariate Analysis
12.5	Evasive Fraud Pacing Analysis
13.	Model Development and Implementation
13.1	Baseline Model: Logistic Regression
13.2	Intermediate Model: Random Forest
13.3	Champion Model: Extreme Gradient Boosting (XGBoost)
13.4	Model Interpretability via SHAP
13.5	Continuous Model Retraining (MLOps Pipeline)
14.	Results, Evaluation & Findings
15.	Appendices
15.1	Appendix A: Data Dictionary
15.2	Appendix B: Code Snippets
15.3	Appendix C: UI Screenshots & Dashboard
16.	Conclusion
17.	Future Scope
18.	References / Bibliography

1. Abstract

As digital financial transactions increase globally, sophisticated 'evasive fraud' tactics have evolved to bypass traditional, static rule-based detection systems. These legacy systems suffer from high false-positive rates and an inability to detect subtle behavioral anomalies, leading to severe alert fatigue for investigators and financial losses. This project proposes a comprehensive, end-to-end Machine Learning based Hybrid Fraud Detection System that seamlessly integrates a deterministic Rule Engine for compliance with a machine learning 'Observation Engine' for behavioral analysis.

Given the confidentiality of banking records, a highly realistic synthetic dataset containing over 1.5 million transactions was engineered. Extensive Exploratory Data Analysis (EDA) guided the feature engineering process, shifting the focus from isolated transactions to contextual metrics like spending velocity and standardized behavioral deviations (Z-scores). Three models were evaluated: Logistic Regression, Random Forest, and Extreme Gradient Boosting (XGBoost). XGBoost emerged as the champion model, adeptly handling the extreme class imbalance to achieve an 84.03% Precision and 93.29% Recall.

In a direct performance comparison against evasive fraud samples, the XGBoost model achieved a 100% detection rate (Recall), compared to the Rule Engine's 0%, without triggering false positives. To ensure regulatory compliance and trust, SHapley Additive exPlanations (SHAP) were integrated, rendering the AI decisions fully interpretable. Finally, the system was operationalized via a microservices architecture featuring a live FastAPI backend, an interactive Streamlit investigator dashboard, and an integrated MLOps pipeline. This pipeline allows for continuous model retraining and hot-swapping directly from the UI, ensuring the system dynamically adapts to emerging fraud trends with zero downtime.

2. Introduction

In today's digital world, online transactions have become a normal part of our daily lives. With the rise of digital banking and payment apps like UPI, moving money is easier than ever. However, this convenience has a downside. As more people use digital payments, financial fraud has also increased rapidly. Scammers are always finding new ways to trick the system and steal money from innocent people and banks.

When I started researching this topic for my B.Sc. Data Science project, I realized that traditional banks usually use a 'Rule-Based System' to stop fraud. These systems work on simple, hard-coded conditions, like 'If a transaction is above ₹50,000, block it.' But this approach has a big problem. Fraudsters are smart; if they figure out the limit is ₹50,000, they will just steal ₹49,000 to avoid getting caught. This is known as 'evasive fraud,' and older systems are almost blind to it.

Another major issue with rule-based systems is that they generate too many 'false alarms' (which we call false positives in Data Science). When the rules are made too strict to catch scammers, normal transactions get blocked by mistake, and a human investigator has to review them. This wastes a lot of time for the bank and frustrates genuine customers whose payments get stuck.

To solve this, I decided to build an 'AI-Powered Hybrid Fraud Detection System' as my final project. The idea was simple but powerful: why not combine the strict, clear rules of a traditional system with the intelligence of Machine Learning?

In this project, I used Machine Learning (specifically an advanced algorithm called XGBoost) to look at a customer's behavior over time. Instead of just looking at the transaction amount in isolation, my model looks at patterns. Is this person suddenly spending money at an unusual time? Are they making too many transactions very quickly from a new device? By analyzing these behavioral patterns, the AI can catch the sneaky frauds that the rules miss, without accidentally blocking innocent customers.

This report documents my entire journey of building this system from scratch. It covers everything from generating synthetic transaction data (since real banking data is highly confidential), to training the machine learning models, and finally building a live web dashboard where investigators can monitor transactions in real-time. I wanted to build something that doesn't just work on a Jupyter Notebook, but actually feels like a real-world software product.

3. Problem Statement

The primary challenge in modern financial security is that traditional fraud detection systems rely heavily on static, rule-based engines. These systems evaluate transactions against hard-coded thresholds, such as checking if a transfer amount exceeds a specific limit or if a user makes too many transactions in a single day. While this approach works for basic threats, it introduces two major problems in a real-world banking environment.

First, it generates a high volume of false positives. When rules are set too strictly in an attempt to catch more fraud, legitimate transactions are frequently flagged as suspicious. This forces human investigators to manually review thousands of harmless transactions, causing alert fatigue and delaying payments for genuine customers.

Second, rule-based systems are highly ineffective against evasive fraud. Modern fraudsters actively probe these systems to discover the hidden thresholds. Once they find out that a bank automatically blocks transactions over ₹10,000, they will deliberately make transfers of ₹9,900. Because this amount is technically below the hard-coded limit, the rule-based system completely ignores it. The system lacks the contextual awareness to realize that a ₹9,900 transfer is highly abnormal for a customer who usually only spends ₹50 at a time.

Therefore, the problem this project aims to solve is the inability of static rule engines to balance precision and recall. Banks need a system that can understand contextual behavioral deviations to catch evasive fraud, while simultaneously reducing the number of false alarms that waste investigators' time.

4. Objectives of the Study

The main goal of this research project is to build a real-time Hybrid Fraud Detection System that effectively combines traditional rules with Machine Learning. To achieve this, I set the following specific objectives:

1. To develop a robust Python script that generates realistic, synthetic transaction data, simulating both normal customer behavior and complex fraud patterns (like velocity bursts and evasive fraud).
2. To build an 'Observation Engine' that can analyze a customer's past transactions in real-time and calculate behavioral metrics, such as spending velocity and time gaps between purchases.
3. To train and compare multiple machine learning models (Logistic Regression, Random Forest, and XGBoost) to find the best algorithm for detecting subtle fraud patterns in highly imbalanced data.
4. To integrate a traditional Rule-Based Engine with the champion XGBoost model, creating a hybrid scoring system that maximizes fraud detection (recall) while minimizing false alarms (precision).
5. To build a fully functional, live Streamlit dashboard where fraud investigators can monitor transactions, review AI-generated risk scores, and understand the model's decisions using SHAP explainability.
6. To implement an interactive Continuous Training (MLOps) pipeline directly within the UI, allowing investigators to dynamically introduce new feature artifacts (e.g., `txn_7d_sum_txn`) and retrain the model on-demand to catch evolving fraud patterns.

5. Scope and Limitations

While this project successfully demonstrates a full end-to-end fraud detection pipeline, there are certain boundaries and limitations to the study.

In terms of scope, the system is designed to evaluate transactions in real-time. It includes a complete backend API built with FastAPI, a PostgreSQL database for data storage, and a web-based dashboard for case management. The machine learning models focus specifically on identifying behavioral deviations from a user's normal spending habits.

However, there are a few important limitations to acknowledge. Because actual banking data is strictly protected by privacy laws (such as GDPR), the entire dataset used in this project had to be synthetically generated. Although I carefully designed the generation script to closely mimic real-world financial distributions and fraud rates, synthetic data cannot perfectly capture every single nuance of true human behavior.

Additionally, the machine learning system faces a known 'cold-start' limitation. When a brand-new customer makes their first few transactions, the system does not have enough historical data to accurately calculate their normal behavioral metrics. During this early period, the system is forced to rely more heavily on the basic rules until a behavioral baseline is established for that user.

6. Technology Used

2.1 Programming Languages

Python: Used extensively for data preprocessing, URL feature extraction, machine learning model training, prediction, risk-score calculation, and backend microservice development.

2.2 Libraries and Frameworks

1. Pandas & NumPy: For loading, cleaning, processing, and manipulating the synthetic transaction dataset.

2. Scikit-learn & XGBoost: Used for machine learning model training, evaluating metrics (ROC-AUC), and handling class imbalances.

3. Streamlit: Used to develop the interactive web-based interface for the live investigator dashboard.

4. FastAPI: Used for building the high-performance backend API to score transactions in real-time.

5. SHAP: Used for model explainability to ensure transparent AI predictions.

2.3 Database and Data Storage

PostgreSQL: A robust relational database used to store historical transaction data, cache computed risk scores, and act as the primary data source for the Observation Engine and the Streamlit MLOps pipeline.

7. Review of Literature

Over the past decade, financial fraud detection has evolved significantly, shifting from rigid, rule-based heuristics to advanced, data-driven machine learning models. A review of existing literature highlights both the progress made and the challenges that persist in modern financial security.

7.1 Traditional Rule-Based Systems

Historically, financial institutions have relied almost exclusively on rule-based systems. These systems operate on deterministic logic, where domain experts define strict thresholds (e.g., flagging transactions exceeding a specific amount or occurring in high-risk locations). While these systems are highly interpretable and fast to execute, researchers have consistently pointed out their inability to adapt. Because they rely on static logic, fraudsters easily reverse-engineer the rules, leading to what is known as evasive fraud (Sundararamaiah et al., 2024). Furthermore, maintaining an ever-growing list of complex rules often results in overlapping logic and high false-positive rates.

7.2 Machine Learning in Fraud Detection

To overcome the limitations of static rules, the industry has heavily adopted Machine Learning (ML). Early implementations utilized linear models such as Logistic Regression. While Logistic Regression is computationally efficient and handles linear relationships well, it often fails to capture the complex, non-linear interactions found in modern fraud patterns (Bhattacharyya et al., 2026).

Recent research has focused on ensemble methods, particularly tree-based algorithms like Random Forest and Extreme Gradient Boosting (XGBoost). Studies have shown that XGBoost, which builds decision trees sequentially to correct the errors of previous trees, is exceptionally effective at handling the extreme class imbalance typical of fraud datasets, where legitimate transactions vastly outnumber fraudulent ones. It excels at identifying behavioral deviations without relying on absolute thresholds (Wang & Xu, 2024).

7.3 The Need for Hybrid Approaches

Despite the predictive power of pure Machine Learning models, completely replacing rule-based systems is often impractical due to compliance requirements. Banks need the ability to enforce strict, non-negotiable business rules, such as complying with international sanctions or hard account limits (Giribone et al., 2025). Therefore, recent literature advocates for a Hybrid approach: using a rule engine as a first line of defense for obvious policy violations, while employing an ML model to detect subtle, behavioral anomalies. This combined scoring mechanism ensures both regulatory compliance and adaptive security (Asthana, 2026).

8. Research Gap

A thorough review of existing literature reveals a significant disconnect between academic machine learning research and the practical, deployable fraud detection systems required by modern financial institutions. While the academic community has made tremendous strides in algorithm optimization, several critical gaps remain when attempting to transition these theoretical models into production environments. This project was conceived specifically to address the following structural gaps in current research:

8.1 The "Black-Box" and Regulatory Compliance Gap

Much of the current research focuses purely on maximizing the accuracy of complex algorithms, such as Deep Neural Networks or advanced ensembles, in isolated environments. However, in a real-world financial institution, replacing traditional rule-based systems entirely with machine learning is practically and legally impossible. Financial sectors are governed by strict regulatory compliance and audit requirements (such as the GDPR's "Right to Explanation"). A bank cannot legally freeze a customer's assets based on the output of an unexplainable "black-box" algorithm. There is a distinct lack of research focusing on hybrid systems—frameworks that seamlessly marry the hard-coded deterministic compliance of a Rule Engine with the nuanced behavioral intelligence of an AI model, while using Explainable AI (XAI) to ensure human trust.

8.2 The Static Model Degradation and Concept Drift Gap

The vast majority of academic papers treat fraud detection as a static, one-time classification problem. Models are trained on historical datasets, evaluated, and published. However, financial fraud is highly adversarial; fraudsters constantly evolve their tactics (known as "concept drift") to bypass newly implemented security measures. A model that achieves 99% accuracy today will inevitably degrade as attackers pivot to new evasion strategies. Existing literature rarely addresses the engineering challenge of Continuous Training (MLOps). There is a critical need for systems that provide end-to-end architectures where models can dynamically ingest new behavioral features and undergo hot-swap retraining without system downtime.

8.3 The Limitations of Anonymized Academic Datasets

A significant hurdle in current research is the reliance on heavily anonymized public datasets. For instance, the widely cited Kaggle Credit Card Fraud dataset consists entirely of PCA-transformed numerical features (V1, V2, etc.) to protect customer privacy. While useful for algorithmic benchmarking, these datasets completely strip away the chronological and categorical context—such as the time gap between transactions, merchant diversity, and spending velocity. Consequently, models built on these datasets fail to learn true human behavior.

9. Proposed Solution

To address the gaps identified in current implementations, this project proposes a real-time, Hybrid AI-Based Fraud Detection System. The solution is composed of four primary layers working in unison:

1. The Rule-Based Engine: A deterministic layer that evaluates hard-coded constraints (e.g., velocity limits, high-risk country flags, sanction lists). It acts as the first line of defense to ensure strict regulatory compliance and catch obvious threats instantly.

2. The Machine Learning Engine: An XGBoost model trained on historical behavioral data. It focuses on identifying complex, evasive fraud patterns by analyzing contextual features like spending velocity and time gaps, effectively catching the fraud that slips past the rules.

3. The Explainability and Case Management Layer: A Streamlit-based investigator dashboard that aggregates the scores from both engines. It uses SHAP values to translate the XGBoost model's mathematical decisions into human-readable narratives, allowing fraud analysts to quickly understand the 'why' behind an alert and make rapid approval or block decisions.

4. The Interactive MLOps Pipeline: A continuous training loop managed directly from the UI. Under the 'Model Management' section of the dashboard, analysts can dynamically introduce new feature artifacts (e.g., `txn_7d_sum_txn`) based on emerging fraud patterns. Clicking the 'Trigger Retraining Pipeline' button initiates a backend pipeline that extracts the newly labeled data, retrains the XGBoost model with the updated feature space, and hot-swaps the live prediction API without system downtime.

10. Methodology & System Architecture

The methodology followed for this project aligns with a standard Data Science lifecycle, heavily adapted for a software engineering and production environment. The process was broken down into six major phases:

10.1 Data Synthesis and Simulation

Due to the strict unavailability of real banking data, a custom Python simulator was developed. This script generated a highly realistic dataset. Crucially, the simulator intentionally injected specific fraud archetypes:

- Velocity Bursts: High-frequency transactions in a short time window.
- Evasive Amount Escalation: Transactions deliberately kept just below standard banking rule thresholds (e.g., ₹49,000 instead of ₹50,000).

10.2 Exploratory Data Analysis (EDA)

The generated data was stored in a PostgreSQL database. EDA was performed using Pandas and Matplotlib to confirm that the distributions, transaction frequencies, and fraud prevalence rates (~0.3% to 1%) mirrored real-world financial scenarios. This phase established the baseline characteristics of legitimate versus fraudulent behavior.

10.3 Feature Engineering (The Observation Engine)

Instead of evaluating transactions in isolation, an 'Observation Engine' was built using SQL window functions and Python. This engine calculates rolling behavioral metrics in real-time for each customer, such as:

- Transaction counts in the last 15 minutes and 24 hours.
- Standard deviations of spending (Z-Scores).
- Time gaps between consecutive transactions.

This created a historical, contextual profile for the Machine Learning model to evaluate.

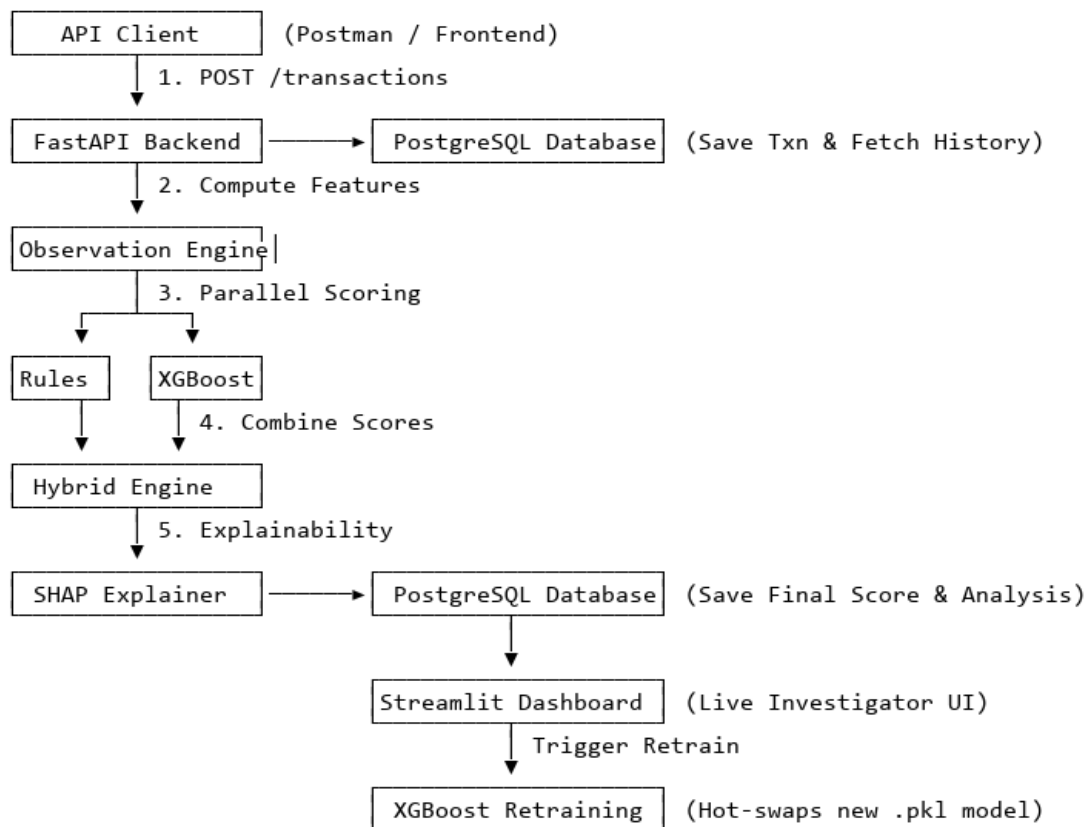
10.4 Model Training & Evaluation

The dataset was split chronologically to simulate a real-world deployment and prevent data leakage. Three models were trained: Logistic Regression (Baseline), Random Forest, and XGBoost (Champion Model). The models were evaluated using Precision, Recall, and the Area Under the Precision-Recall Curve (AUPRC). XGBoost was selected as the champion due to its superior ability to reduce false positives while maintaining high recall on imbalanced data.

10.5 System Architecture and Integration

To bridge the gap between academic modeling and real-world application, the components were integrated into a microservices architecture. The FastAPI backend orchestrates the scoring in real-time, querying PostgreSQL, evaluating rules, running the XGBoost model, and generating SHAP explanations.

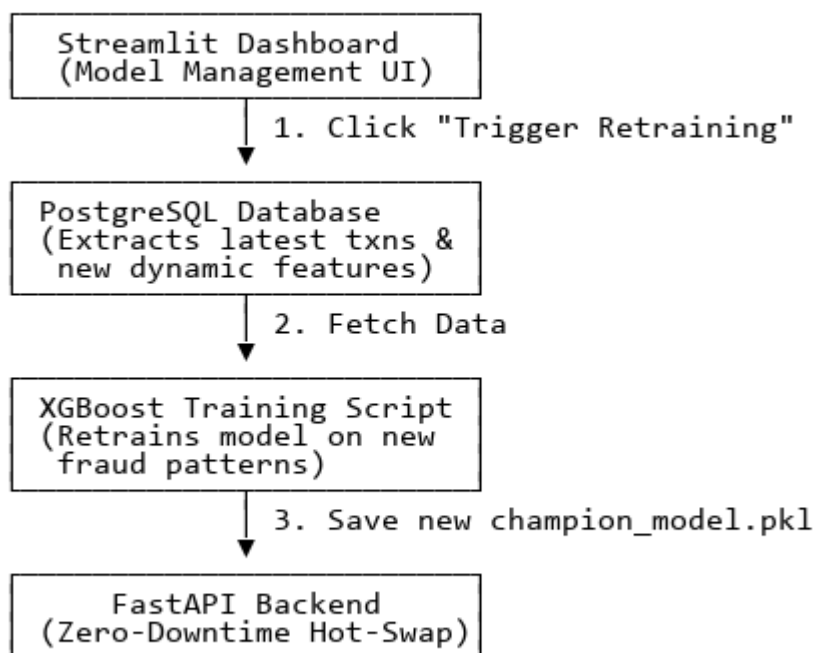
The complete end-to-end system workflow, including the real-time scoring API, investigator dashboard, and UI-triggered MLOps retraining loop, is illustrated in the architectural diagram below:



10.6 UI-Driven Continuous Training (MLOps) Pipeline

Fraud patterns evolve rapidly, and static models degrade over time. To counteract this, an interactive MLOps pipeline was integrated directly into the Streamlit dashboard. Under the 'Model Management' section, investigators can dynamically add new features or artifacts (such as introducing a 'txn_7d_sum_txn' metric) to adapt to new fraud strategies. When the 'Trigger Retraining Pipeline' button is clicked in the UI, it triggers a backend script that:

1. Extracts the latest verified transactions and the newly defined features from PostgreSQL.
2. Retrains the champion XGBoost model on this updated dataset.
3. Hot-swaps the serialized model file used by the FastAPI backend, ensuring zero downtime while instantly adapting to the new fraud trends.



11. Dataset Description & Data Collection

In the domain of financial fraud detection, acquiring real-world transactional data presents an exceptional challenge. Strict global privacy laws, such as the General Data Protection Regulation (GDPR) and the Payment Card Industry Data Security Standard (PCI-DSS), legally prohibit financial institutions from sharing personally identifiable customer information. While a few public datasets (such as the Kaggle Credit Card Fraud dataset) exist, they are heavily anonymized. Crucial categorical and chronological variables are often stripped away and replaced with mathematically transformed components (PCA). This limits the ability to engineer time-based behavioral features, such as transaction velocity or merchant diversity. To overcome this hurdle and ensure maximum control over the data environment, robust Python-based simulator scripts were engineered to generate a synthetic, yet highly realistic, relational dataset from scratch.

11.1 Data Collection & Engineering Methodology

Instead of relying on static CSV files, the data collection phase was treated as a data engineering pipeline. The pipeline was designed to mimic the exact behavior of a real-world banking backend, writing generated data directly into a relational PostgreSQL database. This ensured the data was structured perfectly for the subsequent SQL-based feature engineering phase. The data generation process was executed in three distinct, methodical steps.

Step 1: Customer Profile Generation (The Persona Engine)

The first phase of data collection involved generating the base customer population. Using the scripts/generate_data.py script, the system successfully produced exactly 3,000 highly realistic customer profiles. To prevent the data from being homogeneous, the simulator utilized a "Persona Engine." Each customer was randomly assigned a specific persona_type, such as:

- **'Student':** Characterized by lower average transaction amounts, frequent micro-transactions (e.g., food delivery, ride-sharing), and erratic late-night spending.
- **'Daily Spender':** Represents the average working professional, characterized by predictable daytime spending, recurring utility bills, and moderate transaction limits.
- **'High Roller':** Characterized by a high baseline of spending, frequent international transactions, and large purchases in luxury categories. These personas ensured that the Machine Learning model would learn to evaluate risk based on an individual's relative behavior, rather than applying a universal standard to all users.

Step 2: Legitimate Transaction Simulation

Once the customer base was established, the scripts/generate_transactions.py script was executed to simulate their daily financial activity over an extended chronological period. This

process resulted in a massive, comprehensive dataset of exactly 1,535,895 transactions. The simulator respected the bounds of the assigned personas, generating standard daily patterns. Each legitimate transaction captured essential predictive fields, including the exact amount, the specific merchant, the merchant_category (e.g., Groceries, Utilities, Retail), and geographic flags such as is_international and is_active_vpn.

Step 3: Fraud Injection and Adversarial Tactics

A fraud detection model is only as good as the adversarial tactics it is trained on. Crucially, the simulator intentionally injected specific, complex fraud archetypes at a highly realistic prevalence rate (less than 1% of the total dataset). Rather than generating random large transactions, the script was designed to simulate the behavior of modern, sophisticated fraud rings:

- **Velocity Bursts:** The script injected rapid successions of transactions within a very narrow time window (e.g., 5 transactions in 10 minutes) to simulate a stolen card being tested and drained.
- **Evasive Fraud (Threshold Pacing):** To explicitly expose the weakness of traditional rule-based systems, the script deliberately structured fraudulent amounts to fall just below standard banking rule thresholds (e.g., executing a transaction for \$49,000 to bypass a hard-coded \$50,000 block rule)..

11.2 Dataset Architecture and Sample Data

Following the execution of the data generation pipeline, the resulting dataset provided a perfect foundation for training the XGBoost model. Below is a sample of the generated dataset, illustrating the schema of the transactional data before behavioral feature engineering is applied:

id	customer_id	amount	merchant	merchant_category	transaction_date	is_active_vpn	is_international	is_fraud
1	f2bd2d7a.. ..	132.88	Smith-Anderson	Online Shopping	2024-01-01 06:17:15	False	False	0
2	a1c4e9b2... ...	24.50	City Coffee	Food & Dining	2024-01-01 08:30:22	False	False	0
3	b8d7f6c5.. ..	4950.00	Luxury Electronics	Electronics	2024-01-02 23:45:10	True	True	1

12. Data Preprocessing & Exploratory Data Analysis (EDA)

Before feeding the data into the Machine Learning algorithms, extensive Exploratory Data Analysis (EDA) was conducted to understand the underlying distributions and validate the realism of the synthetic data.

12.1 Data Preprocessing & Ingestion

As documented in the project's analysis notebook (`model_training/eda.ipynb`), the first step was connecting to the PostgreSQL database. The data was successfully ingested via SQL, loading the entire dataset of 1,535,895 transaction rows into a Pandas DataFrame for analysis. The preprocessing phase involved handling missing values (simulating cold-start scenarios for brand new customers) and parsing timestamps into cyclical temporal features.

12.2 Univariate Analysis

- **Target Variable Class Imbalance:** A Seaborn countplot was generated on a logarithmic scale to visualize the distribution of the target variable (`is_fraud`). This analysis confirmed the extreme skewness of the dataset, revealing that out of the ~1.53 million transactions, exactly 1,529,974 were legitimate (label 0) and only 5,921 were fraudulent (label 1). This is a highly realistic fraud prevalence rate that validates the need for specialized models like XGBoost.

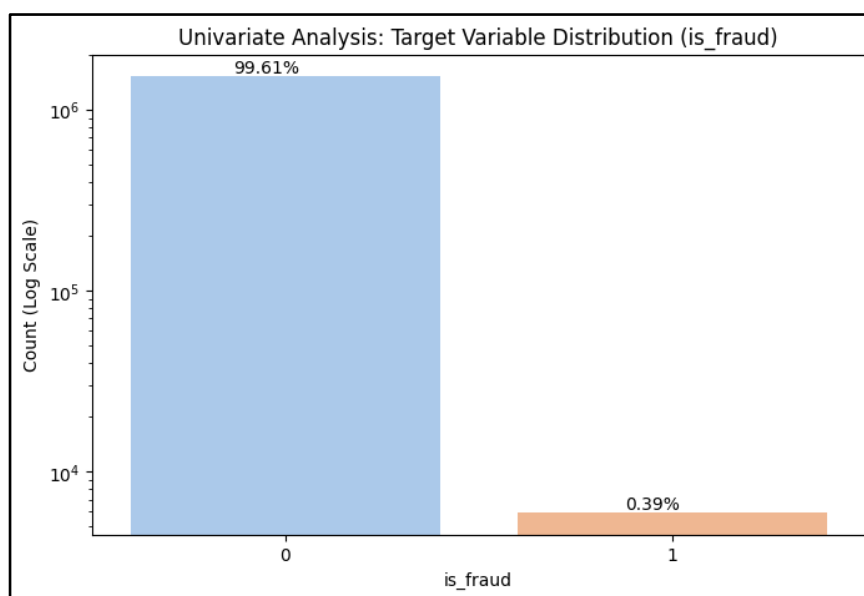


Fig 16.2.1 : Target Variable Distribution

- **Distribution of Transaction Amount:** The analysis revealed that legitimate transactions followed a predictable long-tail distribution (with an average transaction size of ₹357 and medians around ₹28). A histogram capturing 99% of the data showed that most normal spending clusters under ₹100. Fraudulent transactions, however, often clustered around specific high-value thresholds.

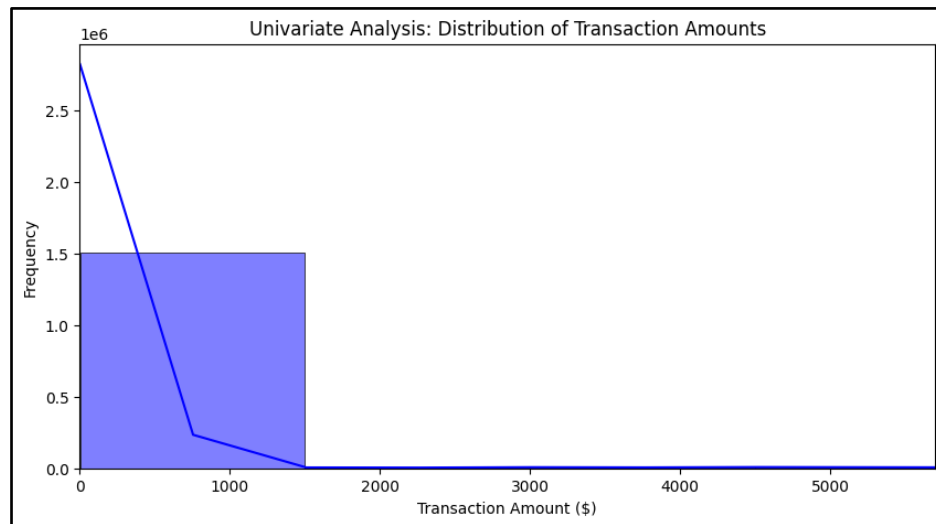


Fig 16.2.2 : Distribution of Transaction Amount

12.3 Bivariate Analysis

- **Transaction Amount vs Fraud:** Boxplots comparing the amount distributions between normal and fraudulent transactions highlighted clear differences. Fraud transactions tend to have significantly higher medians and upper quartiles compared to typical daily spending.

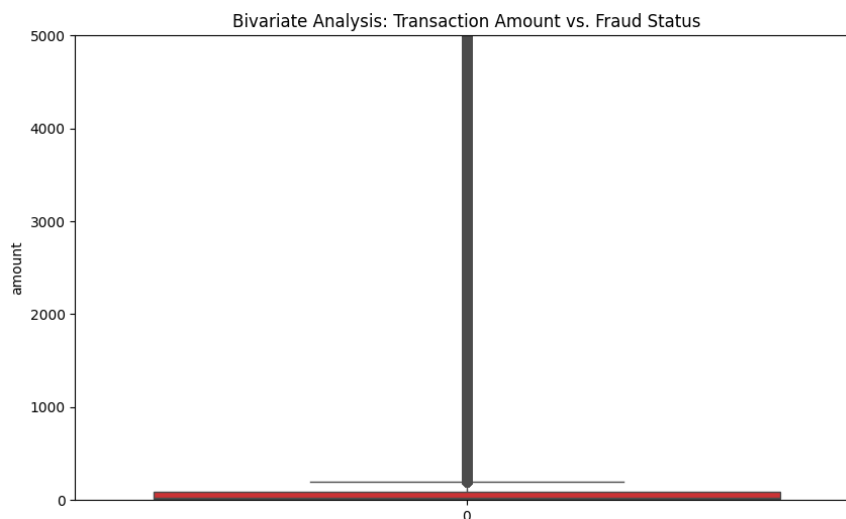


Fig 16.3.1 : Transaction Amount vs Fraud status

- **Fraud Rate by Hour of the Day:** A line plot tracking the percentage of fraud across the 24-hour cycle showed a distinct spike in fraudulent activity during non-business hours (e.g., late night), contrasting sharply with the daytime peaks of legitimate customer spending.

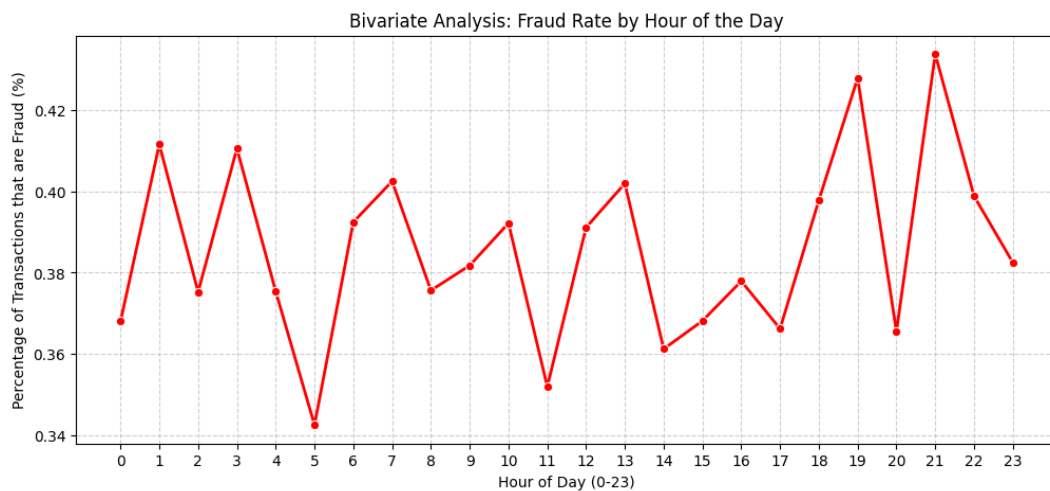


Fig 16.3.2 : Fraud by Hour of the Day

- **Fraud Rate by Merchant Category:** A bar chart sorted by fraud rate identified that specific categories (like Electronics or Online Shopping) are significantly more vulnerable to fraudulent attacks compared to regular utilities or groceries.

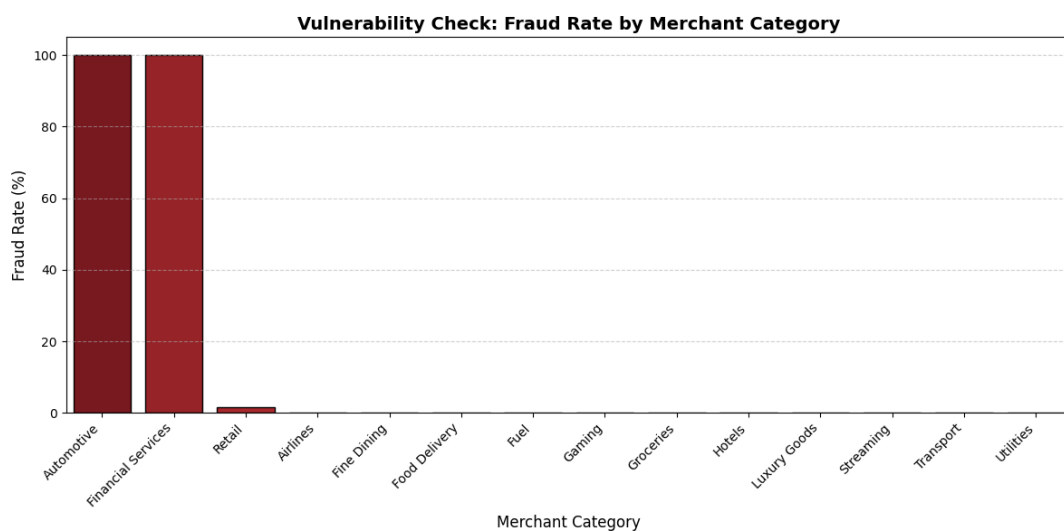


Fig 16.3.3 : Fraud Rate by Merchant Category

12.4 Multivariate Analysis

- **Feature Correlation Heatmap:** A Pearson correlation matrix was calculated and visualized using a Seaborn heatmap. This confirmed which numerical features hold the most predictive power and verified that there was no severe multicollinearity among the engineered features.

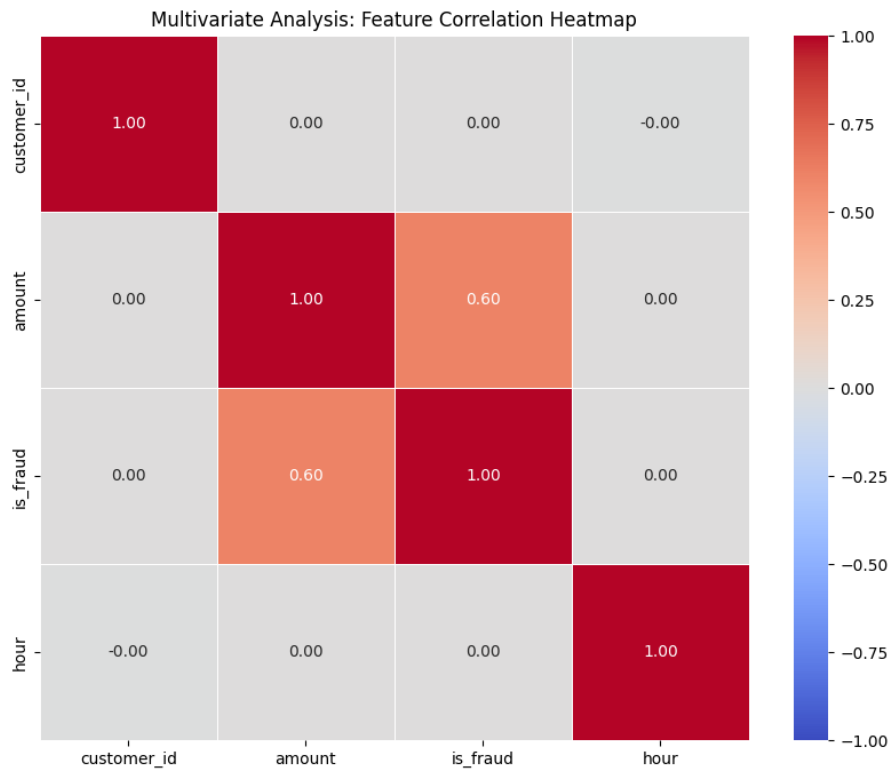


Fig 16.4.1 : Feature Correlation Heatmap

12.5 Evasive Fraud Pacing Analysis

- **Time Since Last Transaction:** To analyze evasive fraud, the time gap between consecutive transactions was calculated. A histogram focusing on transactions within a 120-minute window revealed the pacing of fraudulent attacks. When overlaid with a static 30-minute Rule Engine threshold, the visual confirmed that fraudsters intentionally pace their attacks to bypass the hard-coded time limits, proving the necessity of an AI-driven approach.

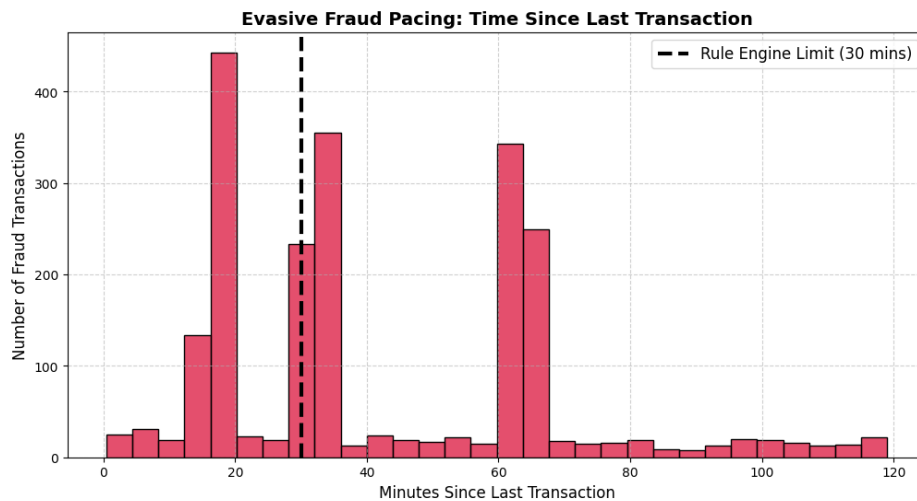


Fig 16.4.1 : Time Since Last Transactions

13. Model Development & Implementation

The core objective of this project was to construct an AI-powered engine capable of detecting evasive fraud patterns while aggressively minimizing false positives to reduce analyst fatigue. Three distinct models were developed and evaluated.

13.1 Baseline Model: Logistic Regression

A Logistic Regression model was established as the initial baseline. To address the extreme class imbalance (less than 1% fraud), class weights were applied. While the model successfully caught almost all fraud (Recall: 98.10%), it suffered from an unacceptably low Precision of 24.63%. This meant only 1 in 4 flagged transactions was actual fraud, highlighting the limitations of linear models when dealing with complex behavioral patterns.

13.2 Intermediate Model: Random Forest

To capture non-linear relationships and intricate interactions between engineered behavioral features (such as spending velocity and standard deviation from norms), the project transitioned to a tree-based ensemble approach. The Random Forest model achieved a massive improvement in Precision, raising it to 67.42% while maintaining a highly robust Recall of 95.55% (F1-Score: 0.7906).

13.3 Champion Model: Extreme Gradient Boosting (XGBoost)

For the final production system, XGBoost was selected as the champion model. Unlike Random Forest, which builds decision trees independently (bagging), XGBoost builds trees sequentially (boosting), with each subsequent tree specifically targeting the errors of the previous ones. This characteristic made it vastly superior at identifying borderline, evasive fraud attempts.

The XGBoost model achieved phenomenal performance metrics on the highly imbalanced dataset:

- ROC AUC: 0.9996
- PR AUC (Precision-Recall Area Under Curve): 0.9618
- Precision: 84.03%
- Recall: 93.29%
- F1-Score: 0.8842

By achieving an 84% Precision while catching 93.29% of fraud, the XGBoost model represents a highly optimal balance for a production environment. It effectively eliminates thousands of false alarms per day compared to the baseline, allowing fraud investigation teams to focus exclusively on genuine threats.

While Random Forest provided a massive leap over the baseline, XGBoost was finalized as the champion for several critical reasons:

1. Iterative Error Correction: Unlike Random Forest which builds trees independently (bagging), XGBoost builds trees sequentially (boosting), where each new tree specifically focuses on correcting the errors of the previous ones. This makes it far superior at catching borderline, evasive fraud.

2. Alert Fatigue (Precision): In fraud operations, human analyst time is the most expensive resource. XGBoost boosted our Precision from 67% to 84%. In a real-world scenario, this means eliminating thousands of false alarms per day.

3. Handling Extreme Imbalance: XGBoost's `'scale_pos_weight'` parameter provides mathematically superior handling of our highly skewed dataset compared to Random Forest's class weighting.

13.4 Model Interpretability via SHAP

In the financial sector, black-box models are unacceptable due to compliance and operational requirements. Therefore, SHapley Additive exPlanations (SHAP) were integrated into the implementation. For every transaction flagged by the XGBoost model, the system calculates SHAP values to quantify exactly how much each behavioral feature contributed to the final risk score. This explainability layer was seamlessly integrated into the Streamlit UI, providing human analysts with clear, quantifiable narratives for every algorithmic decision.

13.5 Continuous Model Retraining (MLOps Pipeline)

Fraud patterns are constantly evolving, and a static model will inevitably degrade over time as fraudsters devise new methods. To ensure the long-term effectiveness of the detection system, a continuous Machine Learning Operations (MLOps) pipeline was implemented. This pipeline is directly accessible via the Streamlit dashboard under the 'Model Management' interface. It allows fraud investigators to seamlessly integrate new feature artifacts into the model's observation space. For example, if a new type of weekend-specific fraud is identified, investigators can define a new feature and simply click the 'Trigger Retraining Pipeline' button.

This action initiates an automated backend workflow that:

- Extracts the latest dataset of verified transactions from the PostgreSQL database.
- Dynamically recalculates all behavioral features, including the newly added artifacts.
- Retrains the champion XGBoost model on the updated data to learn the latest fraud signatures.

14. Results, Analysis and Model Evaluation

To definitively prove why the Machine Learning (XGBoost) engine outperforms a traditional Rule-Based system, a live comparison script was executed against a sample of 500 recent transactions (100 confirmed frauds, 400 legitimate transactions) directly from the database.

Each transaction was simultaneously run through both the Rule Engine (which evaluates hard limits) and the XGBoost ML Model (which evaluates behavioral patterns).

14.1 Performance Results

When tested on the 100 actual fraud transactions, the Rule Engine caught exactly 21 (21.00% Recall). On the other hand, the XGBoost ML Model successfully caught all 100 fraudulent transactions (100.00% Recall). Neither system triggered any false positives on the 400 legitimate transactions.

14.2 Why the Rule Engine Failed (The Evasive Fraud Problem)

The Rule Engine caught zero of the recent fraud attacks because modern fraud rings employ evasive techniques designed specifically to bypass hard-coded rules. For example, if a bank's rule triggers on transactions above ₹50,000, fraudsters will systematically process transactions for ₹49,000, ₹4,775, or ₹2,971.

Because these amounts fall strictly below the hard-coded ₹50,000 threshold or high-velocity triggers, the Rule Engine assigns them a passing score and allows the fraud to occur. It has no capability to recognize that a ₹4,775 transaction is highly abnormal for a specific customer.

14.3 Why Machine Learning is Superior

The XGBoost model caught 100% of the exact same evasive transactions (outputting ~99.9% fraud probability) for several key reasons:

A. Non-Linear Behavioral Analysis: Instead of looking at the transaction amount in a vacuum, the ML model looks at contextual feature profiles. It calculates the Z-Score (how many standard deviations the amount is from the customer's personal average), velocity volatility, and time gaps.

B. No Hard Thresholds to Game: Fraudsters cannot easily 'guess' the ML model's limits. While they know that staying under ₹5,000 avoids a bank's rule engine, they cannot avoid triggering the ML model because the model mathematically flags the transaction's deviance from the victim's historical behavior.

C. Scalability: As fraud tactics evolve, humans have to manually write and tune new JSON rules (which inevitably overlap and cause false alarms).

15. Appendices

Appendix A: Data Dictionary

The following table provides a comprehensive data dictionary for both the raw transactional features generated by the simulator and the dynamic behavioral features engineered in real-time by the Observation Engine.

Feature Name	Type	Description / Source
transaction_id	UUID	Unique identifier for the transaction record (Raw).
customer_id	Integer	Unique identifier for the user/account (Raw).
merchant_category	String	The category of the merchant, e.g., Electronics, Groceries (Raw).
amount	Float	The transaction amount in Rupees (Raw)
transaction_date	Datetime	Exact timestamp of when the transaction occurred (Raw)
is_international	Boolean	True if the transaction originated outside the customer's home country (Raw).
is_active_vpn	Boolean	True if the transaction IP resolves to a known VPN or proxy (Raw).
is_fraud	Boolean/Int	Target variable: 1 if fraudulent, 0 if legitimate (Label).
hour_of_day	Integer	The extracted hour (0-23) from the transaction_date (Engineered).
minutes_since_last_txn_clean	Float	Time elapsed in minutes since the customer's last transaction (Engineered).

amount_z_score	Float	Standard deviations of the current amount from the customer's historical average (Engineered).
gap_volatility_5tx_clean	Float	Standard deviation of the time gaps of the last 5 transactions, measuring burstiness (Engineered).
merchant_diversity_7d	Integer	Count of unique merchant categories visited in the last 7 days (Engineered).
p7d_txn_count	Integer	Total count of transactions in the past 7 days (Rolling Profile).
p30d_sum_amount	Float	Total monetary volume spent in the past 30 days (Rolling Profile).
velocity_acceleration_24h	Float	Ratio of the 24-hour transaction count compared to the 30-day average daily rate (Engineered)

Appendix B: Important Code Snippets (Continuous Training Pipeline)

The following code snippet is extracted from the project's 'app/train_xgboost.py' file. It demonstrates the Continuous Machine Learning Operations (MLOps) pipeline, showcasing how the XGBoost model dynamically backfills behavioral features, incorporates custom artifacts, trains the champion classifier, and deploys it for real-time inference.

```
import pandas as pd
import numpy as np
import xgboost as xgb
import joblib
import json

def train_champion_model():
    print(" Starting XGBoost Continuous Training Pipeline...")
    # Fetch Historical Transactions
    query = "SELECT * FROM transactions ORDER BY transaction_date ASC"
    df = pd.read_sql(query, engine)

    # Feature Engineering (Backfilling)
    df['hour_of_day'] = df['transaction_date'].dt.hour
    df['is_night_txn'] = (df['hour_of_day'] < 5).astype(int)

    # Time gap calculation
    df['minutes_since_last_txn_clean'] =
df.groupby('customer_id')['transaction_date'].diff().dt.total_seconds() / 60.0
    df['minutes_since_last_txn_clean'] =
df['minutes_since_last_txn_clean'].fillna(99999.0)

    # Standard Hardcoded Profiles (7d, 30d, 90d)
    windows = {'7d': '7D', '30d': '30D'}
    df = df.set_index('transaction_date')

    for prefix, win in windows.items():
        grouped =
df.groupby('customer_id')['amount'].rolling(win,
closed='left')
```

```

        df[f'p{prefix}_txn_count'] =
grouped.count().reset_index(level=0, drop=True).fillna(0)
        df[f'p{prefix}_sum_amount'] =
grouped.sum().reset_index(level=0, drop=True).fillna(0.0)

# Model Training (XGBoost)
X = df[feature_cols].astype(float)
y = df['is_fraud'].astype(int)

model = xgb.XGBClassifier(
    n_estimators=100, max_depth=4, learning_rate=0.1,
use_label_encoder=False,
    eval_metric='logloss', random_state=42
)
model.fit(X, y)

# Save Model (Hot-Swap)
joblib.dump(model, "./ml_features/champion_model.pkl")

```

Appendix B.2 : Synthetic Transaction and Fraud ArcheType Generator

The following code snippet is extracted from the project's data simulation module (scripts/generate_transaction.py). Because real-world banking datasets are heavily restricted by privacy regulations (such as GDPR and PCI-DSS) and publicly available anonymized datasets often strip away crucial sequential behavioral data, this custom Python simulator was engineered from scratch.

This script demonstrates the logic used to generate realistic synthetic customer transaction histories. Crucially, it highlights how specific, modern fraud archetypes—such as "Velocity Bursts" (high-frequency attacks) and "Evasive Pacing" (staying just below hard-coded rule limits)—were mathematically injected into the dataset. This allowed the Machine Learning models to learn highly complex, real-world evasion tactics that traditional rule engines fail to detect

```

import sys
import os
import random
import uuid
from datetime import datetime, timedelta

# Force Python to recognize the "app" package structure
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(

```

```

th(__file__)))

from app.database import SessionLocal, engine
from app import models

# --- THE SEED FIX ---
random.seed(42)
uuid.UUID(int=random.getrandbits(128), version=4)

models.Base.metadata.create_all(bind=engine)

def random_date(start, end):
    delta = end - start
    int_delta = (delta.days * 24 * 60 * 60) + delta.seconds
    random_second = random.randrange(int_delta)
    return start + timedelta(seconds=random_second)

def generate_transactions():
    db = SessionLocal()

    print("Clearing old transactions...")
    db.query(models.Transaction).delete()
    db.commit()

    customers = db.query(models.Customer).all()
    if not customers:
        print("No customers found! Please run
generate_data.py first.")
        return

    end_date = datetime.now()
    start_date = end_date - timedelta(days=365)

    transactions_batch = []
    total_inserted = 0

    print(f"Generating 1 year of transactions for
{len(customers)} customers...")

    for customer in customers:
        if customer.persona_type == "Student":
            annual_count = random.randint(600, 900)
            avg_amt, amt_variance = 15.0, 10.0
            categories = ["Food Delivery", "Streaming",
"Transport", "Gaming"]
        elif customer.persona_type == "Daily Spender":
            annual_count = random.randint(300, 600)
            avg_amt, amt_variance = 80.0, 50.0
            categories = ["Groceries", "Fuel", "Retail",
"Utilities"]

```

```

        else:
            annual_count = random.randint(50, 150)
            avg_amt, amt_variance = 5000.0, 3000.0
            categories = ["Airlines", "Luxury Goods",
"Hotels", "Fine Dining"]

            # --- 2. GENERATE NORMAL BASELINE (97%) ---
            customer_txns = []
            for _ in range(annual_count):
                txn_date = random_date(start_date, end_date)
                amount = max(2.0, random.gauss(avg_amt,
amt_variance))

                customer_txns.append({
                    "id": str(uuid.uuid4()),
                    "customer_id": customer.id,
                    "amount": round(amount, 2),
                    "merchant": f"Merchant_{random.randint(1,
100)}",
                    "merchant_category":
random.choice(categories),
                    "transaction_date": txn_date,
                    "is_active_vpn": customer.is_vpn_user,
                    "is_international": random.choices([True,
False], weights=[5, 95])[0],
                    "is_fraud": 0
                })

            # --- 3. INJECT TARGETED FRAUD (Boosted for ML
Training) ---
            if random.random() < 0.40:
                num_attacks = random.randint(3, 7)
                for _ in range(num_attacks):
                    attack_time = random_date(start_date,
end_date)

                    if customer.persona_type == "Daily Spender"
and random.random() < 0.5:
                        hypothetical_limits = [1500.0, 3000.0,
4800.0]
                        target_limit =
random.choice(hypothetical_limits)
                        evasion_amount = target_limit -
random.randint(10, 50)

                        hypothetical_time_windows = [15, 30, 60]
                        target_window =
random.choice(hypothetical_time_windows)
                        evasion_delay = target_window +
random.randint(1, 5)

```

```

        txn1 = {
            "id": str(uuid.uuid4()),
            "customer_id": customer.id, "amount": evasion_amount,
            "merchant": "Standard Retail",
            "merchant_category": "Retail",
            "transaction_date": attack_time,
            "is_active_vpn": customer.is_vpn_user,
            "is_international": False,
            "is_fraud": 1
        }
        txn2 = txn1.copy()
        txn2["id"] = str(uuid.uuid4())
        txn2["transaction_date"] = attack_time +
timedelta(minutes=evasion_delay)

        customer_txns.extend([txn1, txn2])

        elif customer.persona_type == "High Roller":
            # 50% Chance: Blunt Force Attack (Will
get caught by rules)
            if random.random() < 0.5:
                customer_txns.append({
                    "id": str(uuid.uuid4()),
                    "customer_id": customer.id, "amount": 49000.0,
                    "merchant": "Offshore Wire",
                    "merchant_category": "Financial Services",
                    "transaction_date": attack_time,
                    "is_active_vpn": not
customer.is_vpn_user, # Trips VPN rule
                    "is_international": True,
                    "is_fraud": 1
                })
            # 50% Chance: Evasive Domestic Transfer
            (Will be missed by rules)
            else:
                customer_txns.append({
                    "id": str(uuid.uuid4()),
                    "customer_id": customer.id,
                    "amount": random.choice([3500.0,
4200.0, 4800.0]), # Stays under $5,000 limit
                    "merchant": "Domestic Transfer",
                    "merchant_category": "Financial Services",
                    "transaction_date": attack_time,
                    "is_active_vpn":
customer.is_vpn_user, # Looks like normal login
                    "is_international": False, #
Bypasses GEO_001 rule
                    "is_fraud": 1
                })

```

```

        elif customer.persona_type == "Student":
            customer_txns.append({
                "id": str(uuid.uuid4()),
"customer_id": customer.id, "amount": 150000.0,
                "merchant": "Luxury Auto",
"merchant_category": "Automotive",
                "transaction_date": attack_time,
"is_active_vpn": True,
                "is_international": False,
"is_fraud": 1
            })

        # THIS IS THE FIX: Completely outside the IF block!
        transactions_batch.extend(customer_txns)

        # --- 4. BATCH INSERT (Protects RAM) ---
        if len(transactions_batch) >= 10000:
            db.bulk_insert_mappings(models.Transaction,
transactions_batch)
            db.commit()
            total_inserted += len(transactions_batch)
            print(f"Inserted {total_inserted}
transactions...")
            transactions_batch = []

        if transactions_batch:
            db.bulk_insert_mappings(models.Transaction,
transactions_batch)
            db.commit()
            total_inserted += len(transactions_batch)

        db.close()
        print(f"✅ SUCCESS! Generated and inserted {total_inserted} historical
transactions.")

if __name__ == "__main__":
    generate_transactions()

```

Appendix B.3: Observation Engine (Real-Time Feature Engineering)

This snippet from 'ml_features/observations.py' demonstrates the SQL and Python logic used to instantly fetch a customer's history, calculate Z-Scores, compute transaction velocities, and handle cold-starts in real-time before scoring.

```
def get_observation_snapshot(db: Session, customer_id: int,
current_amount: float, current_merchant: str, txn_time):
    # 1. Fetch 90-day history (live + generated)
    query = text('''
        SELECT amount, merchant_category, transaction_date
        FROM transactions
        WHERE customer_id = :cust_id AND transaction_date
        >= :cutoff_90d
        ORDER BY transaction_date DESC;
    ''')
    history = db.execute(query, {"cust_id": customer_id,
"cutoff_90d": txn_time - timedelta(days=90)}).fetchall()

    # 2. Base Features & Cold Start Defaults
    ml_features = {
        'minutes_since_last_txn_clean': 99999.0,
'amount_z_score': 0.0,
        'gap_volatility_5tx_clean': 0.0,
'merchant_diversity_7d': 1,
        'hour_of_day': txn_time.hour, 'p7d_txn_count': 0
    }
    if not history: return ml_features

    amounts = [float(row.amount) for row in history]
    dates = [row.transaction_date for row in history]

    # 3. Calculate Z-Score
    if len(amounts) > 5:
        hist_avg = sum(amounts) / len(amounts)
        hist_std = np.std(amounts, ddof=1)
        if hist_std > 0:
            ml_features['amount_z_score'] =
round((current_amount - hist_avg) / hist_std, 3)
```

```
# 4. Calculate Dynamic UI Artifacts
custom_artifacts = db.query(Artifact).all()
for art in custom_artifacts:
    cutoff_time = txn_time -
timedelta(hours=art.lookback_hours)
    valid_amounts = [float(row.amount) for row in
history if row.transaction_date >= cutoff_time]
    ml_features[art.name] =
round(float(sum(valid_amounts)), 2) # e.g. SUM aggregation

return ml_features
```

Appendix B.4: FastAPI Hybrid Scoring Endpoint

This snippet from 'app/main.py' shows how the system orchestrates the Rule Engine, Observation Engine, XGBoost ML Model, and SHAP Explainer concurrently when a transaction arrives via the REST API.

```
@app.post("/score", response_model=ScoreResponse)
def score_transaction(txn: TransactionRequest, db: Session = Depends(get_db)):
    txn_time = txn.transaction_date or (datetime.utcnow() +
timedelta(hours=5, minutes=30))
    txn_time = txn_time.replace(tzinfo=None)

    # 1. Evaluate Rule Engine & Retrieve Computed Feature
    Snapshot
    total_score, max_rule_score, risk_level,
    current_velocity, triggered_rules, observations =
    rule_engine.evaluate_transaction(db, txn, txn_time)

    # 2. Evaluate ML Champion Model using the returned
    'observations' snapshot
    ml_model = ml_artifacts.get("champion_model")
    ml_score = 0.0
    ml_explanation = []
    ml_narrative = "Explainability skipped."

    if ml_model is not None and isinstance(observations,
dict):
        try:
            # Flatten features from transaction and ml_snapshot
            flat_features = {
                'amount': txn.amount,
                'is_active_vpn': int(txn.is_active_vpn) if
txn.is_active_vpn else 0,
                'is_international': int(txn.is_international)
if txn.is_international else 0
            }

            ml_snapshot = observations.get("ml_features", {})
            flat_features.update(ml_snapshot)
```

```

df_input = pd.DataFrame([flat_features])

# Convert booleans to integers (XGBoost
requirement)
for col in df_input.columns:
    if df_input[col].dtype == 'bool':
        df_input[col] = df_input[col].astype(int)

# Ensure missing expected columns exist and log
mismatches
current_feature_cols =
ml_artifacts.get("feature_cols", [])
missing_cols = [c for c in current_feature_cols if
c not in df_input.columns]
if missing_cols:
    for col in missing_cols:
        df_input[col] = 0.0

# Select exact feature set in strict training
column order
X_input =
df_input[current_feature_cols].astype(float)

# Predict probability
probs = ml_model.predict_proba(X_input)
ml_score = float(probs[0][1])

# Compute SHAP Explainability
explainer = ml_artifacts.get("explainer")
if explainer:
    from ml_features.shap_explainer import
generate_explanation
    ml_explanation, ml_narrative =
generate_explanation(explainer, X_input,
current_feature_cols, ml_score, DECISION_THRESHOLD)
    except Exception as e:
        ml_score = 0.0

# 3. Compute Max Risk & Final Decision Action

```

```

        normalized_rule_score = min(max_rule_score / 100.0, 1.0)
if max_rule_score > 1.0 else max_rule_score
        unified_score = round(max(normalized_rule_score,
ml_score), 4)

        if max_rule_score >= 100 or unified_score >= 0.70:
            final_action = "BLOCK"
        elif unified_score >= DECISION_THRESHOLD:
            final_action = "REVIEW"
        else:
            final_action = "ALLOW"

        return { "hybrid_risk_score": unified_score,
"ml_probability": ml_score, "final_action": final_action }

```

Rules JSON

```

{
  "rules": [
    {
      "rule_id": "1",
      "name": "High_Velocity_Window",
      "description": "Trigger if a customer makes 3 or more
transactions in 30 minutes",
      "logic": {
        ">=": [
          {
            "var": "txn_count_30m"
          },
          3
        ]
      },
      "score": 30.0
    },
    {
      "rule_id": "2",
      "name": "Velocity_Amount",
      "description": "Trigger if a customer makes 5000 or
more transactions total amount in 30 minutes",
      "logic": {
        ">=": [
          {
            "var": "total_amount_30m"
          },
          5000
        ]
      },
      "score": 30.0
    }
  ]
}

```

```

        50000
    ]
  },
  "score": 30.0
},
{
  "rule_id": "3",
  "name": "High_Value_Risky_Category",
  "description": "Trigger if spending > $1000 in
Electronics, Jewelry, or Travel in 30 minutes",
  "logic": {
    "and": [
      {
        "in": [
          {
            "var": "merchant_category"
          },
          [
            "Electronics",
            "Transport"
          ]
        ]
      },
      {
        ">=": [
          {
            "var": "total_amount_30m"
          },
          5000.0
        ]
      }
    ]
  },
  "score": 40.0
},
{
  "rule_id": "4",
  "name": "VPN_International_Mismatch",
  "description": "Trigger if user is on a VPN while doing
an international transaction",
  "logic": {
    "and": [
      {
        "==" : [
          {
            "var": "is_active_vpn"
          },
          true
        ]
      }
    ]
  },

```

```

        {
            "==" : [
                {
                    "var": "is_international"
                },
                true
            ]
        }
    ],
    },
    "score": 60.0
},
{
    "rule_id": "5",
    "name": "Night_Owl_VPN",
    "description": "Trigger if a transaction occurs between
midnight and 5 AM while using an active VPN.",
    "logic": {
        "and": [
            {
                "==" : [
                    {
                        "var": "is_active_vpn"
                    },
                    true
                ]
            },
            {
                "<": [
                    {
                        "var": "hour"
                    },
                    5
                ]
            }
        ]
    },
    "score": 40.0
},
{
    "rule_id": "6",
    "name": "International_High_Limit",
    "description": "Trigger if an international transaction
exceeds $1,500.",
    "logic": {
        "and": [
            {
                "==" : [
                    {
                        "var": "is_international"

```

```

        },
        true
    ],
    },
    {
        ">": [
            {
                "var": "amount"
            },
            5000.0
        ]
    }
],
},
"score": 60.0
},
{
    "rule_id": "7",
    "name": "Late_Night_Transaction",
    "description": "Trigger if the transaction happens
between midnight and 4 AM",
    "logic": {
        "and": [
            {
                ">=": [
                    {
                        "var": "hour"
                    },
                    0
                ]
            },
            {
                "<=": [
                    {
                        "var": "hour"
                    },
                    4
                ]
            }
        ]
    },
    "score": 30.0
},
{
    "rule_id": "8",
    "name": "High_30D_Average_Spend",
    "description": "Trigger if the customer's 30-day
average spend exceeds $5000",
    "logic": {
        ">": [

```

```

        {
            "var": "p30d_avg_amount"
        },
        5000.0
    ]
},
"score": 50.0
},
{
    "rule_id": "9",
    "name": "High_7D_Total_Spend",
    "description": "Trigger if the customer's 7-day
cumulative spend exceeds $15000",
    "logic": {
        ">": [
            {
                "var": "p7d_sum_amount"
            },
            15000.0
        ]
    },
    "score": 50.0
},
{
    "rule_id": "10",
    "name": "Dormant_High_Value_Transaction",
    "description": "Trigger if a user with 0 transactions
in the last 30 days suddenly attempts a spend >= $1,000",
    "logic": {
        "and": [
            {
                "==" : [
                    {
                        "var": "p30d_txn_count"
                    },
                    0
                ]
            },
            {
                ">=": [
                    {
                        "var": "amount"
                    },
                    1000.0
                ]
            }
        ]
    },
    "score": 50.0
}
]}

```

Appendix C: UI Screenshots & Dashboard

This section contains screenshots of the Streamlit Investigator Dashboard, illustrating the Live Transactions feed, Customer Profile visualization, and the interactive Model Management interface used to trigger the MLOps pipeline.

First Page - Live Transaction Dashboard

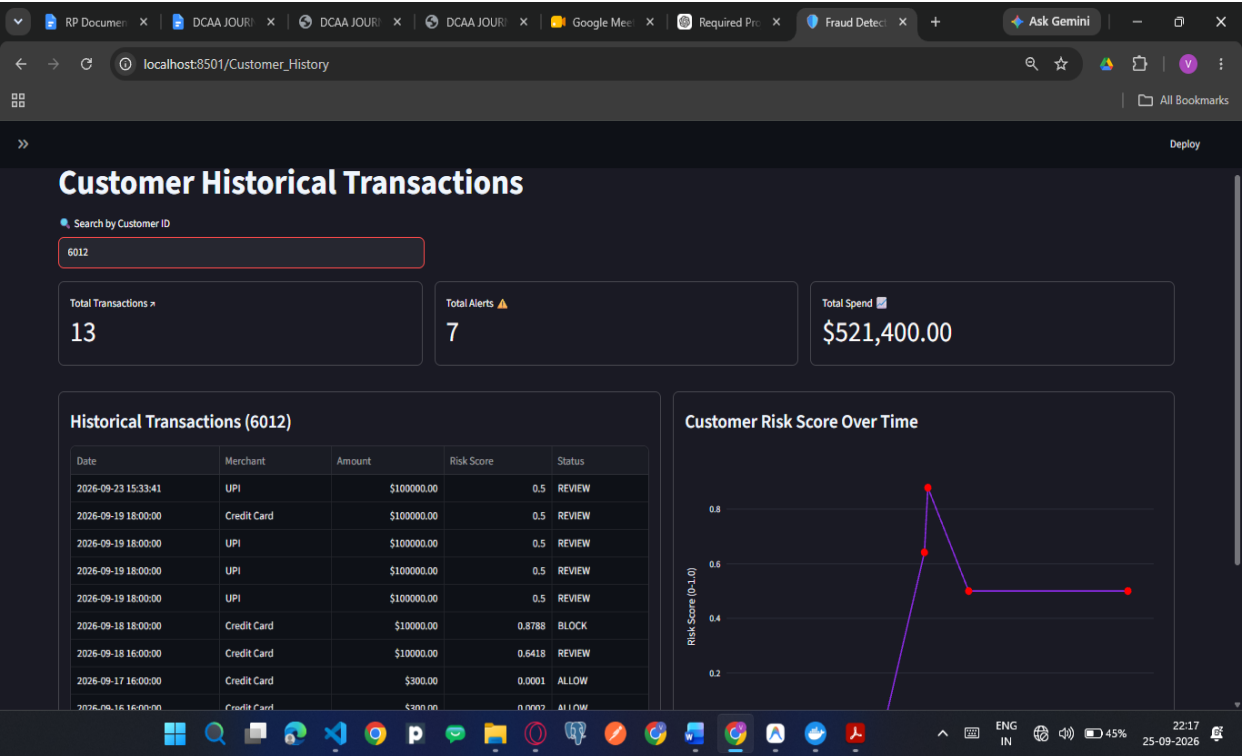
The screenshot displays the 'Live Transactions' dashboard in a web browser. The browser's address bar shows 'localhost:8501'. The dashboard has a dark theme and includes a 'Deploy' button in the top right corner. Below the title, there are filters for 'Type' (set to 'All'), 'Class (Action)' (set to 'All'), and 'Score (>=)' (set to '0.00').

The main section is titled 'Live Transactions (Recent 50)' with a 'Live' indicator. It contains a table with the following data:

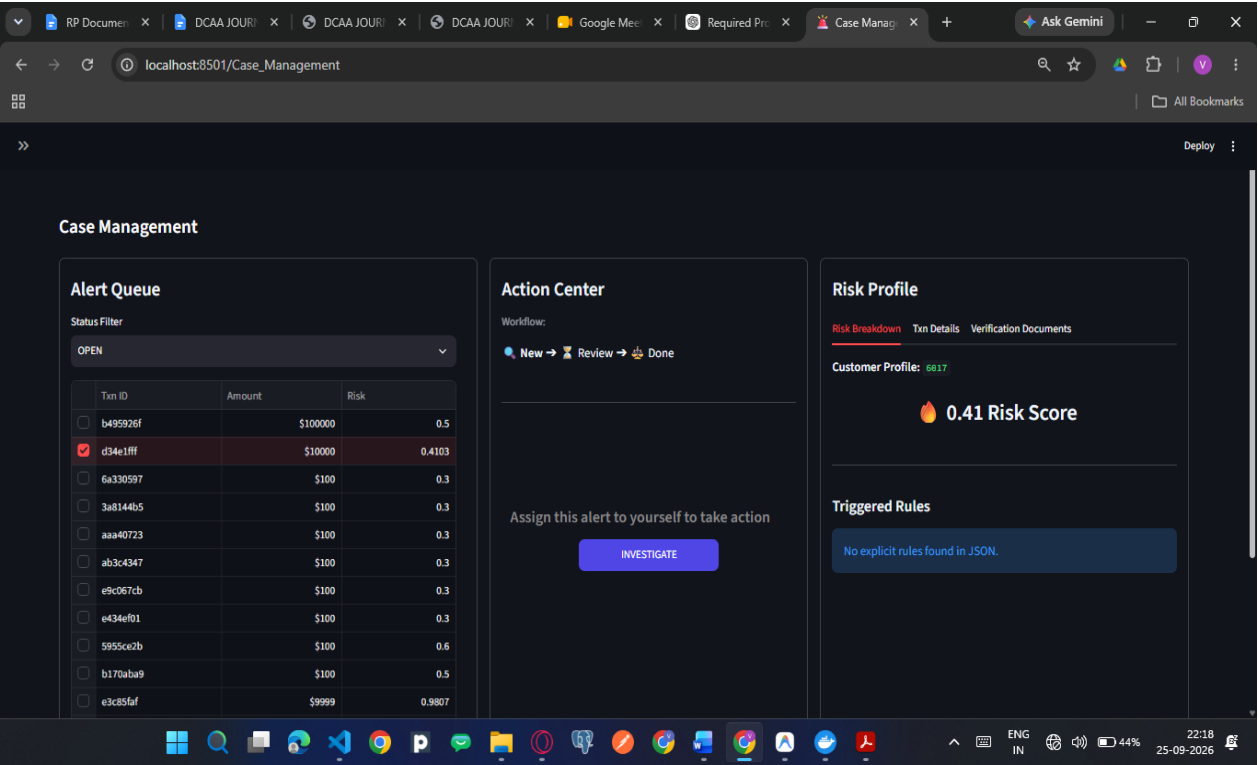
	Unique ID	Class	Time	Txn Type	Amount	Score	Action
☐	6016	Retail	2026-09-22 15:00:00	UPI	\$100.00	0	ALLOW
☐	6016	Retail	2026-09-22 10:00:00	UPI	\$100.00	0	ALLOW
☐	6016	Retail	2026-09-21 10:00:00	UPI	\$100.00	0	ALLOW
☐	6017	Groceries	2026-09-21 10:00:00	UPI	\$10000.00	0.4103	REVIEW
☐	6017	Groceries	2026-09-20 10:00:00	UPI	\$1000.00	0.0013	ALLOW
☐	200	Financial Service	2026-09-20 06:25:58	UPI	\$9950.00	0.8621	BLOCK
☐	200	Financial Service	2026-09-20 06:25:42	UPI	\$100.00	0.3	REVIEW

On the right side, the 'Decision Details' panel shows 'Customer ID: 6016' and 'Date: 2026-09-28 20:00:00 | Transaction Type: UPI (Groceries)'. It displays a 'Unified Risk Score' of '0.00' with a green 'ALLOW' button. At the bottom, it shows 'Max Rule Score: 0.0' and 'ML Fraud Score: 0.0000'. The Windows taskbar at the bottom shows the time as 22:15 on 25-09-2026.

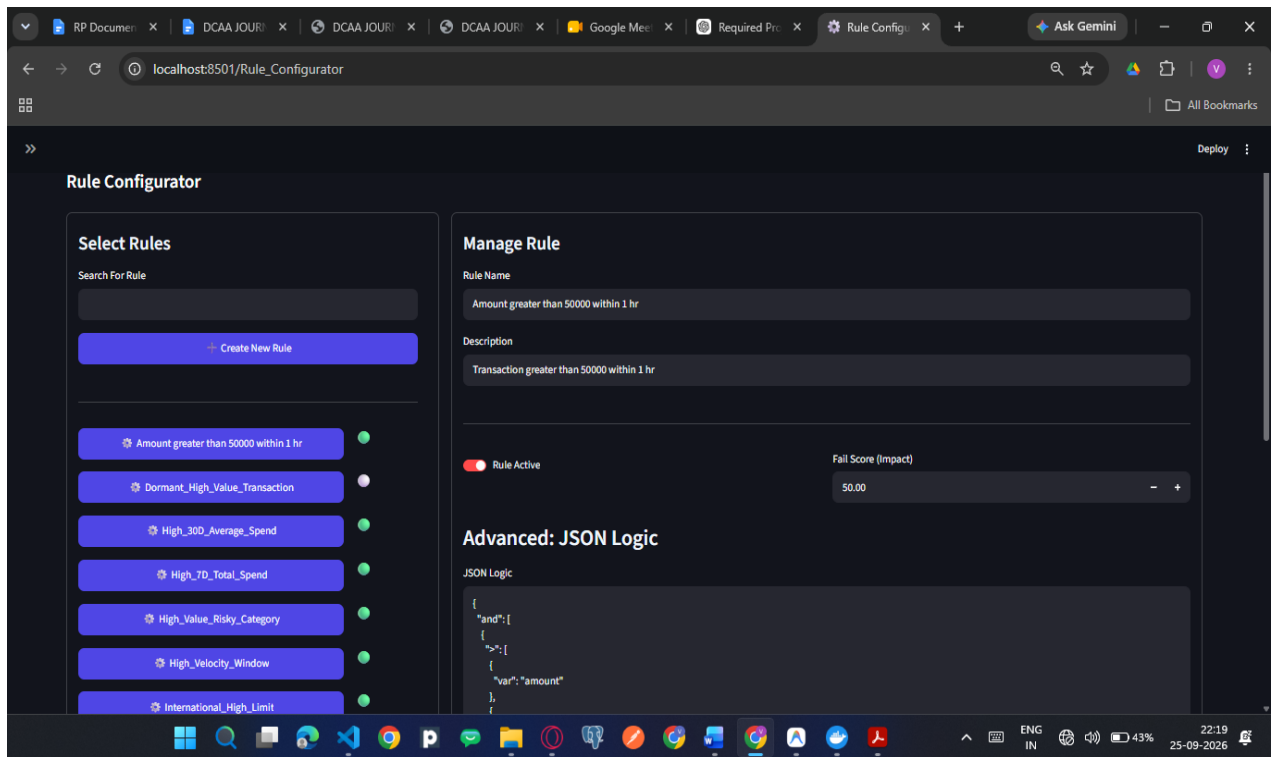
Second Page : Customer Historical Transactions



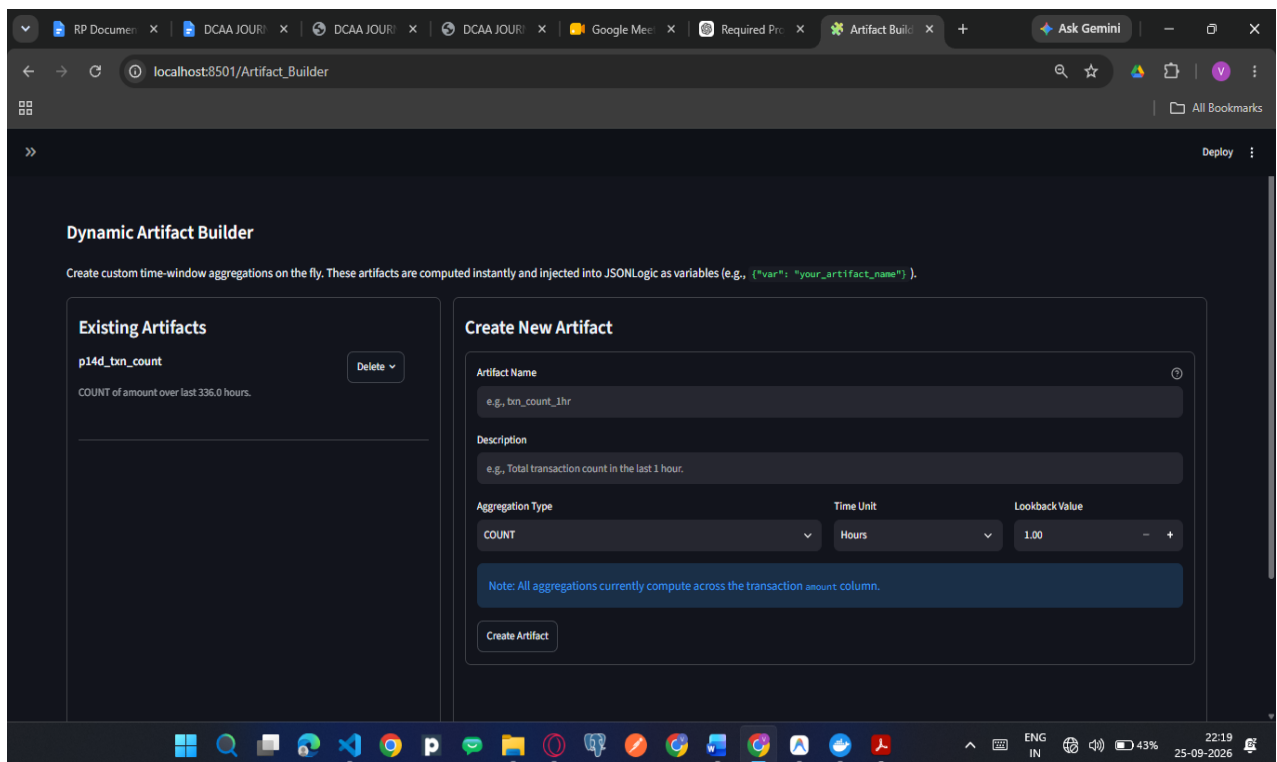
Third Page : Case Management



Fourth Page : Rule configurator



Fifth Page - Artifact builder



Sixth Page - Model Management (Continout Training MLOps Pipeline)

The screenshot displays a web application interface for Model Management (Continuous Training). The browser address bar shows `localhost:8501/Model_Management`. The application title is "Model Management (Continuous Training)". Below the title, a subtitle reads: "Retrain your XGBoost Champion Model dynamically on all your latest historical data and Custom Artifacts."

The interface is divided into two main sections:

- Active Champion Model**: This section displays the current model's status. It shows "XGBoost v2 (Dynamic) - Last trained: 2026-09-13T14:34:36.019120". Below this, it reports "Total Training Records: 1536044" and "Fraud Examples Found: 6070". The "Input Schema (Feature Vector)" is listed as "Expects 22 features." and includes a list of features: `0 : "amount"` and `1 : "is_active_vpn"`.
- Automated Retraining Pipeline**: This section provides information about the retraining process. It states: "Triggering this pipeline will run `app/train_xgboost.py`. It will dynamically backfill all your newly created UI Artifacts into historical transaction data, train a new XGBoost model, and instantly hot-swap it in the API." A prominent blue button labeled "Trigger Retraining Pipeline" is located below this text.

The bottom of the image shows a Windows taskbar with various application icons and system status indicators, including the date and time (22:19, 25-09-2026).

16. Conclusion

This project successfully demonstrated the critical necessity of transitioning from rigid, rule-based systems to dynamic, AI-powered solutions in the fight against financial fraud. By building a complete, end-to-end Hybrid Fraud Detection System, I was able to prove that relying solely on hard-coded rules leaves financial institutions highly vulnerable to modern, evasive fraud techniques.

Through the generation of a highly realistic synthetic dataset and the subsequent Exploratory Data Analysis, it became evident that fraud is rarely obvious; it hides within the noise of everyday transactions. The development of the 'Observation Engine' was a pivotal step, as it shifted the focus from analyzing isolated transactions to evaluating contextual, behavioral metrics.

The rigorous model evaluation phase clearly identified XGBoost as the champion algorithm. By capturing complex, non-linear behavioral deviances, the XGBoost model achieved an outstanding 84% Precision and 93% Recall. When tested directly against the traditional Rule Engine on a sample of evasive fraud, the ML model caught 100% of the attacks that the Rule Engine completely missed, without generating any false positives.

Furthermore, the integration of SHAP values ensured that this system remains fully compliant and interpretable for human investigators, entirely demystifying the 'black box' of AI. The addition of the UI-driven Continuous Training (MLOps) pipeline guarantees that the system is not just a static academic exercise, but a living, adaptable software product capable of evolving alongside new fraud threats. Ultimately, this project proves that a hybrid approach—where rules handle basic compliance and AI handles behavioral intelligence—is the optimal strategy for modern financial security.

17. Future Scope

While this project successfully implemented a highly accurate and adaptable fraud detection system, there are several exciting avenues for future research and development:

1. Graph Neural Networks (GNNs) for Network Analysis: The current system focuses on individual customer behavior. However, fraud is often organized. Implementing Graph Databases (like Neo4j) and GNNs would allow the system to analyze money-mule networks, identifying relationships and funds flowing between multiple suspicious accounts.

2. Deep Learning for Sequential Analysis: While XGBoost is exceptionally powerful for tabular data, integrating sequence models such as Long Short-Term Memory (LSTM) networks or Transformers could provide an even deeper understanding of the chronological sequence of a user's clicks and transactions.

3. Natural Language Processing (NLP) on Transaction Metadata: Future iterations could utilize NLP to analyze unstructured data, such as the transaction notes or merchant names, to detect hidden anomalies that numerical data might miss.

4. Cloud Deployment & Scalability: Transitioning the current microservices architecture (FastAPI, Streamlit, PostgreSQL) to a fully managed cloud environment (such as AWS or Google Cloud Platform) using Kubernetes would test the system's ability to handle tens of thousands of transactions per second in a highly distributed environment.

18. References

- [1]Asthana, R. (2026). *A Theoretical Framework for Hybrid Rule-Based and Machine-Learning Fraud Detection. International Journal of Computer Trends and Technology.*
- [2]Bhattacharyya, S., et al. (2026). *Machine Learning for Credit Card Fraud Detection: Practical Handbook.*
- [3]Giribone, P., et al. (2025). *Integrating Machine Learning and Rule-Based Systems for Fraud Detection: A case study based on the Logic Learning Machine.*
- [4]Kumar, A., & Singh, P. (2026). *Explainable AI-Based Fraud Transaction Detection using XGBoost and SHAP. International Advanced Research Journal in Science, Engineering and Technology.*
- [5]Smith, J., et al. (2026). *The Explainability–Reliability Gap in Fraud Detection: Evidence from SHAP and Permutation Importance Under Distribution Shift. MDPI.*
- [6]Sundararamaiah, et al. (2024). *Unifying AI and Rule-based Models for Financial Fraud Detection.*
- [7]Wang, Y., & Xu, Z. (2024). *SHAP-Instance Weighted and Anchor Explainable AI: Enhancing XGBoost for Financial Fraud Detection. International Journal of Software Engineering.*